Employee Raise Issue using ServiceNow -
Capstone Project Documentation

1. Introduction

Project Title: Employee-Raise-Issue-Record-Producer-Service-Portal-Integration-Guide

2. Project Overview

Purpose: To let employees raise issues through a ServiceNow Service Portal and have every issue saved as a proper record in a custom table.

Features: Custom scoped application, custom table with auto-numbered records, UI policies, dependent Category and Subcategory choices, Record Producer, Service Portal with two custom widgets.

This project is an Employee Raise Issue solution that I built on ServiceNow. It gives employees one place to report an issue, and every report is captured in the same structured way.

I started by creating a scoped application called *Employee Center*. Inside it I created a custom table, *Employee raises issue*, with an auto-generated number and fields for Requester, Category, Subcategory, Short description, State, Priority and Assignment group.

To keep the data clean, I added two UI policies. One makes Short description mandatory when the State is New, and the other makes the Number field read-only. I also set the Subcategory field to depend on Category, so Network shows VPN, Hardware shows Laptop, Software shows Server and Access shows Forgot password.

Employees submit an issue through a Record Producer named *Raise Employee Issue*. It has five questions, and each one is mapped to a field of the table, so submitting the form creates the record directly.

For the front end I created a Service Portal called *Requesting portal* (URL suffix *employee_request*). Its home page has two widgets side by side: a Commercial widget that describes the services, and a Link redirect widget with a *Raise Ticket* button that opens the Record Producer.

Overall, the project shows how a Record Producer and a Service Portal can be put together into one issue-reporting flow in ServiceNow.

3. Architecture

Frontend: ServiceNow Service Portal (Requesting portal, Employee Center Portal Page, Commercial widget, Link redirect widget)

Backend: UI Policies, dictionary dependency, Record Producer variable mapping, widget client script

Database: Custom table *Employee raises issue* inside the scoped application *Employee Center*

I kept the solution in three layers so that the screen the employee sees, the form logic and the stored data stay separate.

1. Presentation Layer (Frontend)

The Requesting portal uses the Employee Center homepage layout and the Employee Center Coral Theme. The portal page holds a 6 + 6 container with the two widgets. The client script of the Link

redirect widget takes the employee to the Record Producer using its sys_id.

2. Service Layer

The Record Producer *Raise Employee Issue* sits in the Service Catalog under the Service category. It asks for the requester, category, subcategory, state and short description.

3. Data Layer

All issues are stored in the *Employee raises issue* table. Requester points to the User table and Assignment group points to the Group table.

4. Business Rules on the Form

Two UI policies and the Category-to-Subcategory dependency control how the fields behave on the table form.

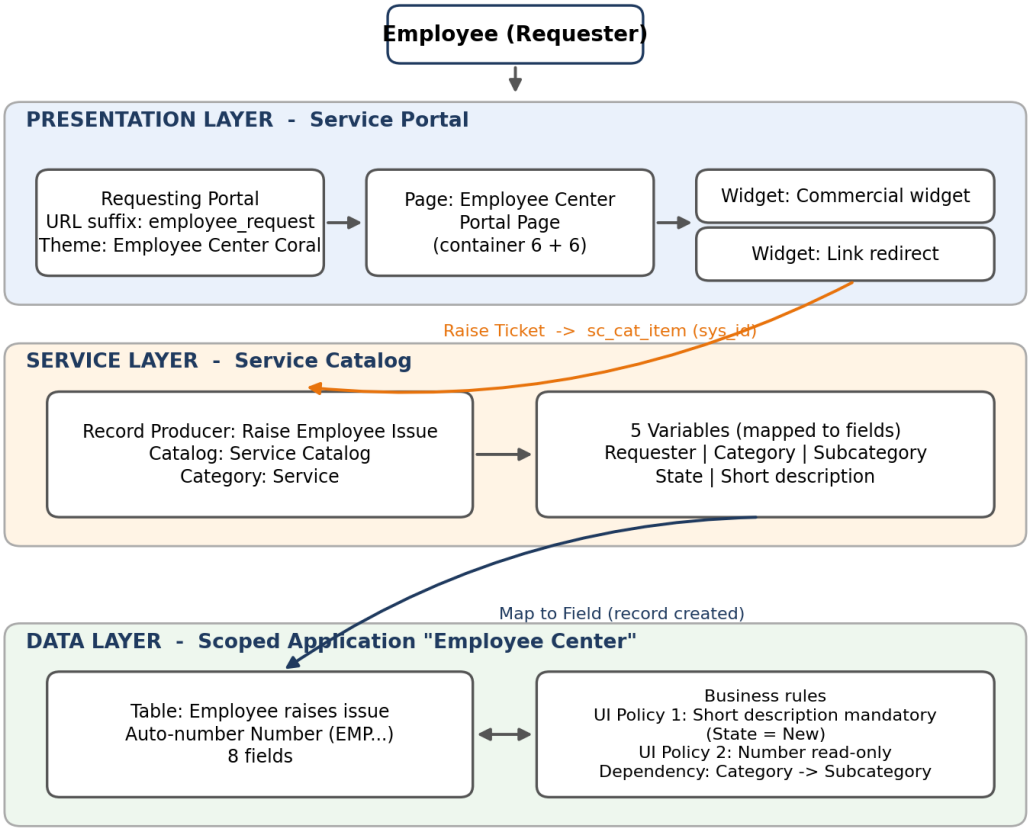


Figure 1: Architecture of the Employee Raise Issue solution

4. Setup Instructions

Prerequisites: ServiceNow Developer Instance

Installation: Create the application and table, add the UI policies and dependency, create the Record Producer, then build the portal, page and widgets.

Step 1: Create the Application

- Navigate to All → App Engine → ServiceNow Studio
- Click Create → App
- Name: Employee Center, Scope: Scoped, then Continue

- Go back to the home page and reload; it shows as the current application

Step 2: Create the Table and Fields

- Navigate to System Definition → Tables and click New
- Label: Employee raises issue, and enable Auto number under Controls
- Open Show Form → Configure → Form Design and add the fields:
- – Number (Auto-number), Requester (Reference to sys_user)
- – Category (Choice: Network, Hardware, Software, Access)
- – Subcategory (Choice: VPN, Laptop, Server, Forgot password)
- – Short description (String), State (Choice: New, In progress, Resolved, Closed)
- – Priority (Choice: Critical, High, Medium, Low), Assignment group (Reference to sys_user_group)

Step 3: Create the UI Policies

- Navigate to All → UI Policies → New, with the table set to Employee raises issue
- Policy 1: condition State is New; UI Policy Action on Short description with Mandatory = True
- Policy 2: no condition; UI Policy Action on Number with Read only = True

Step 4: Create the Category-Subcategory Dependency

- Right-click Subcategory → Configure Dictionary → Advanced view
- Set the Dependent field to Category
- In the Choices list, enter the backend value of the matching Category for each Subcategory choice

Step 5: Create the Record Producer

- Navigate to Service Catalog → Catalog Definitions → Record Producers → New
- Name: Raise Employee Issue, Table: Employee raises issue, Catalog: Service Catalog, Category: Service
- Add a picture and set the Meta name to the table name, then save
- Add five variables with Map to Field enabled: Requester name, Category, Subcategory, State, Short description (orders 100, 200, 300 and so on)

Step 6: Create the Service Portal and Page

- Service Portal Configuration → New: Title Requesting portal, URL suffix employee_request, Homepage ec_standard_home, KB home page esc_knowledge_home, Login page login, Main menu Employee Center Menu, 404 page 404, Theme Employee Center Coral Theme
- Service Portal → Pages → New: Title Employee Center Portal Page, ID employee_center_portal_page

Step 7: Create the Widgets

- Widget Editor → Create a new widget: Commercial widget (HTML and CSS)
- Widget Editor → Create a new widget: Link redirect (HTML, CSS and client script)
- In the client script, paste the sys_id of the Record Producer:

```
function ($scope, $location) {
  var c = this;
  var RECORD_PRODUCER_SYS_ID = 'ae2cd61183433210de6fc590ceaad3a4';

  c.openRecordProducer = function () {
    $location.url('sp?id=sc_cat_item&sys_id=' + RECORD_PRODUCER_SYS_ID);
  };
}
```

Step 8: Add the Widgets to the Page

- Service Portal Configuration → Designer, and open the Employee Center homepage
- Create a 6 + 6 container, then drag the Commercial widget to the left and the Link redirect widget to the right

Step 9: Test the System

- Open the portal URL and click Raise Ticket
- Submit the form and check the record in the table
- Check the field behaviour set by the UI policies and the dependency

Final Note

Once all the components are configured, the portal can be opened at the *employee_request* URL of the instance and the whole flow can be tested.

5. Folder / Component Structure

1. Application

- Scoped application *Employee Center* that holds the components

2. Data Structure

- Table *Employee raises issue* with auto-number
- Fields: Number, Requester, Category, Subcategory, Short description, State, Priority, Assignment group

3. Form Logic

- UI policy: Short description mandatory when State is New
- UI policy: Number field read-only
- Dictionary dependency: Subcategory depends on Category

4. Service Catalog

- Record Producer *Raise Employee Issue* with five variables mapped to table fields

5. Service Portal

- Portal *Requesting portal* (employee_request)
- Page *Employee Center Portal Page*
- Widgets: *Commercial widget* (left) and *Link redirect* (right)

Key Structural Principle

- UI Layer → Service Portal + widgets

- Service Layer → Record Producer
- Logic Layer → UI policies + dependency
- Data Layer → Employee raises issue table

6. Running the Application

Step 1: Access the Service Portal

I open the portal at *https://<instance>.service-now.com/employee_request*. The home page shows My Recent Items, including *Raise Employee Issue*, and further down the two widgets.

Step 2: Open the Form

Clicking *Raise Ticket* on the right widget opens the *Raise Employee Issue* Record Producer.

Step 3: Fill the Form

- Requester name
- Category (Network, Software, Hardware, Access)
- Subcategory (VPN, Laptop, Server, Forgot password)
- State of the issue
- Short description

Step 4: Submit

On Submit, the answers are mapped to the fields of the *Employee raises issue* table and a new record is created with an automatic number.

Step 5: Check the Record

Opening the record in the table shows the submitted values, and the field rules from the UI policies can be seen on the form.

7. API / Automation Logic

This project uses ServiceNow's own configuration features and does not call any external API. The only external item is the image URL used as the background of the Link redirect widget.

1. Record Producer Mapping

Each variable of the Record Producer has Map to Field enabled, so the answers go straight into the matching table field.

2. Client Script Redirect

The *Raise Ticket* link calls *openRecordProducer()*. The function builds the catalog item URL with the sys_id of the Record Producer and opens it.

3. Form Rules

The UI policies and the Category-to-Subcategory dependency apply on the table form.

4. Data Flow Summary

Employee clicks Raise Ticket → client script opens the Record Producer → employee submits the form → variables are mapped to table fields → record is created with an auto-generated number.

8. Authentication

The portal and the form run on the ServiceNow instance, so I used the login handled by the platform. In the Service Portal settings the login page is set to *login* and the 404 page is set to *404*. The Record Producer screenshot shows the form opened by a logged-in user (System Administrator). The steps in this project do not include custom roles or ACLs.

9. User Interface

1. Portal Home Page

The Requesting portal has a logo, a top menu, a Recent Items section with *Raise Employee Issue* and a search bar.

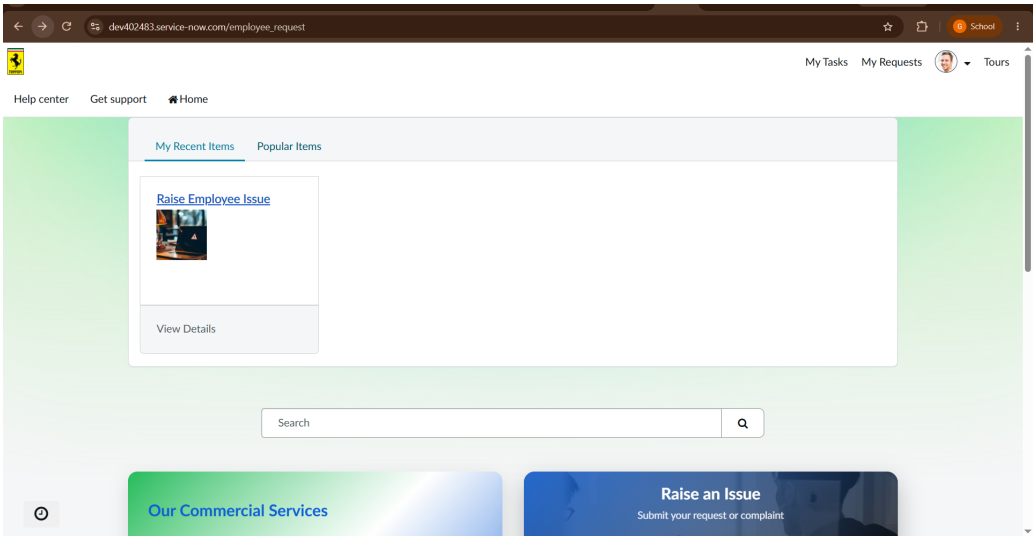


Figure 2: Requesting portal home page

2. Widgets

The Commercial widget (green, white and blue gradient) lists the services. Next to it, the Raise an Issue widget has an image background and a Raise Ticket button. Quick links and My active items appear below them.

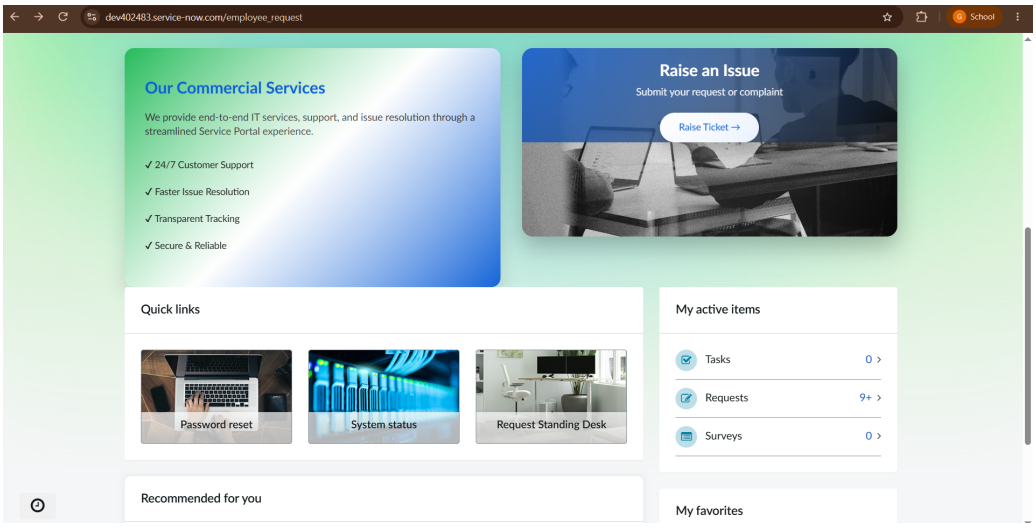


Figure 3: Commercial widget and Raise an Issue widget

3. Raise Employee Issue Form

The Record Producer form shows the picture, the requester field, the Category options and the Subcategory options, with Submit and Save as Draft buttons.

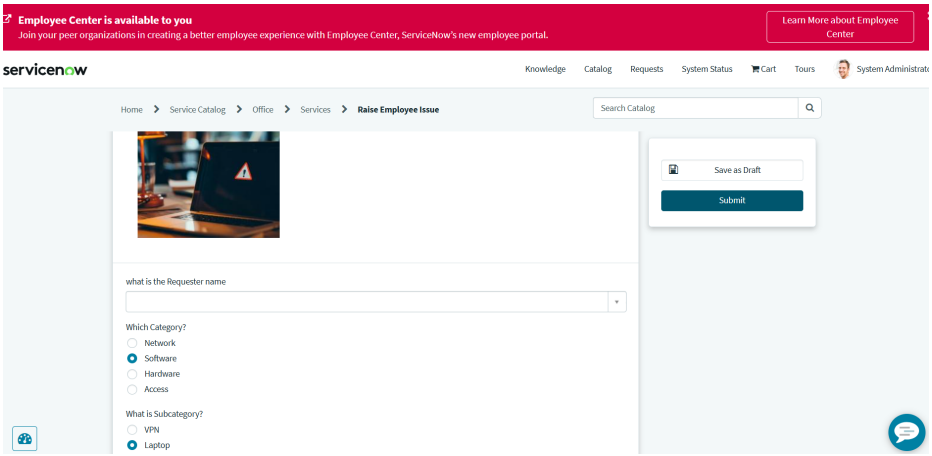


Figure 4: Raise Employee Issue form

4. Record in the Table

After submission, the record (EMP0001009) opens in the *Employee raises issue* form with the Number greyed out.

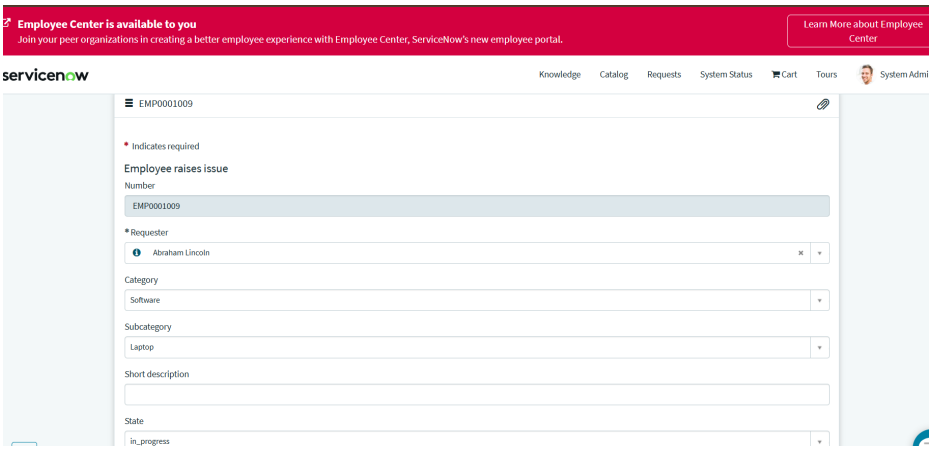


Figure 5: Created record in the Employee raises issue table

10. Testing

I tested the solution by going through it manually, from the portal URL to the record in the table.

1. Test Scope

- Service Portal home page and widgets
- Record Producer form
- Record creation in the table
- Number field behaviour

2. Test Scenarios and Cases

Test Case 1: Portal Access

• Input: Open the employee_request URL
• Expected Output: Requesting portal loads with logo and Recent Items
• Result: Pass

Test Case 2: Widgets
• Input: Scroll down the portal home page
• Expected Output: Commercial widget and Raise an Issue widget are shown side by side
• Result: Pass

Test Case 3: Raise Ticket Link
• Input: Click Raise Ticket
• Expected Output: Raise Employee Issue form opens
• Result: Pass

Test Case 4: Form Fields
• Input: Open the form
• Expected Output: Requester, Category and Subcategory options are shown with Submit and Save as Draft
• Result: Pass

Test Case 5: Record Creation
• Input: Fill in the form and click Submit
• Expected Output: Record is created in Employee raises issue with an auto-generated number
• Result: Pass

Test Case 6: Number Field
• Input: Open the created record
• Expected Output: Number field is read-only
• Result: Pass

11. Known Issues

1. Subcategory Options on the Portal Form

I set the Category-to-Subcategory dependency on the table field. On the Record Producer form, all Subcategory options are shown, and my screenshots show Software selected with Laptop.

2. Record Producer sys_id in the Widget

The Link redirect widget has the sys_id of the Record Producer written in its client script, so the value has to be changed if the Record Producer is created again.

3. External Image URL

The background image of the Link redirect widget is loaded from an external URL, so it needs internet access to appear.

4. Priority and Assignment Group

These two fields exist in the table, but the Record Producer does not ask for them.

12. Future Enhancements

1. Dependent Subcategory on the Record Producer

Apply the same Category-to-Subcategory dependency to the Subcategory variable, so the portal form shows only the matching option.

2. Priority and Assignment Group on the Form

Add Priority and Assignment group to the Record Producer so they are filled in at submission.